

Async JavaScript — Promises

How do you deal with things that haven't happened yet?

Today's agenda

1. Opening problem
2. What do you already know?
3. Core concepts — Promises
4. Live coding — articles API
5. Exercises
6. Reflection

What should this code do?

```
fetch("https://articles-zxjs.onrender.com/articles")
  .then(function(response) {
    response.json().then(function(list) {
      fetch("https://articles-zxjs.onrender.com" + list.data[0])
        .then(function(r) {
          r.json().then(function(article) {
            console.log("First article: " + article.data.title
              + " by " + article.data.author);
          });
        });
    });
  });
});
```

The question

If the first request fails — or the second fetch never resolves — where does the error go?

Think for 2 minutes. Then discuss with the person next to you.

What do you already think?

True, False, or Not sure?

1. JavaScript executes line by line — async functions always wait for each other.
2. Callbacks are the only way to handle async code.
3. If an async operation fails inside a `setTimeout`, the error crashes the page.

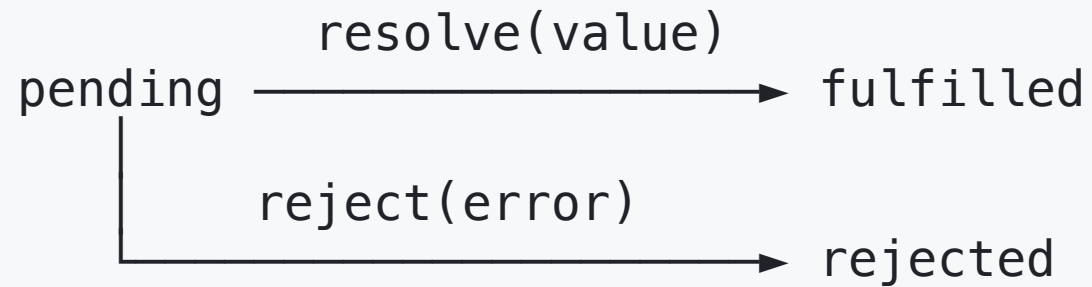
A Promise is

An object that represents the **eventual result** of an async operation.

Like a receipt from an online order:

- You don't have the package yet
- But you have a confirmed reference to it
- You can already decide what to do when it arrives — or if it doesn't

Promise states



- Starts as **pending**
- Settles exactly **once** — either fulfilled or rejected
- Can never go back to pending

Creating a Promise

```
let promise = new Promise(function(resolve, reject) {  
  setTimeout(() => resolve("done!"), 1000);  
});
```

The executor runs immediately.

Call `resolve` when it works. Call `reject` when it doesn't.

Consuming a Promise

```
fetch("https://articles-zxjs.onrender.com/articles/1")  
  .then(response => response.json())  
  .then(data => console.log(data.data.title + " by " + data.data.author))  
  .catch(error => console.log("Error:", error))  
  .finally(() => console.log("Done."));
```

- `.then()` — runs on success
- `.catch()` — runs on failure
- `.finally()` — runs either way

Chaining

Each `.then()` returns a **new Promise**.

```
fetch("https://articles-zxjs.onrender.com/articles/1")  
  .then(response => response.json())    // returns a Promise  
  .then(data => data.data)              // returns the article object  
  .then(article => console.log(article.title + " by " + article.author));
```

The result passes forward through the chain — flat, readable, no nesting.

This is NOT chaining

```
// ❌ Both handlers receive the original response – not chained  
promise.then(handler1);  
promise.then(handler2);
```

```
// ✅ Each handler receives the result of the previous one  
promise.then(handler1).then(handler2);
```

Where does `.catch()` go?

```
fetch(url)
  .then(response => response.json())
  .then(data => {
    throw new Error("Something broke");
  })
  .then(result => console.log(result)) // ← skipped
  .catch(error => console.log("Caught:", error.message));
```

`.catch()` catches errors from **anywhere above it** in the chain.

Live demo — reading an article

```
const baseUrl = "https://articles-zxjs.onrender.com";

fetch(baseUrl + "/articles")
  .then(response => response.json())
  .then(list => fetch(baseUrl + list.data[0]))
  .then(response => response.json())
  .then(article => {
    console.log(`${article.data.title} by ${article.data.author}`);
  })
  .catch(error => console.log("Error:", error.message))
  .finally(() => console.log("Request complete."));
```

Exercise time

Open VS Code. Work individually.

Start with Exercise 1 in `exercises.md` :

Fetch article 1 and log its title and author to the console.

Starter code is in the exercises file.

Stuck? Log `data` first and explore the response structure.

Exercises 2 & 3

Exercise 2: Fetch the article list, then use the result to fetch the first article — two requests chained in sequence.

Exercise 3: Pick any article ID (1–10). Add `.finally()`.

Reflection

Take 3 minutes — write your answers, then we share.

1. What specifically makes the Promise chain better than the nested callbacks from the opening?
2. Where can you place `.catch()` in a chain — and does it matter?
3. What from today connects to something you already knew?
4. What are you still uncertain about?

After this lesson

Complete **Exercises 4–6** at home:

- **Ex 4:** Fetch three articles at once with `Promise.all()`
- **Ex 5:** Break things on purpose — observe how errors propagate
- **Ex 6:** Rewrite with callbacks — then argue which you'd use in a real project

Use `quiz.md` to check your own understanding.

See you next time

We start with your questions from the reflection.